SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1710**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I ***Madhusarvesh.S.K (192525318)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1. AI-Powered Emergency Medicine Delivery Drones

i. Greedy Best-First Search (GBFS): behaviour, strengths and weaknesses

GBFS selects the frontier location with the smallest heuristic value, here the straight-line distance to the medicine destination: **f(n) = h(n)**. It repeatedly chooses the point that appears closest to the goal. This can produce a quick route when the heuristic is informative and the map is stable. However, straight-line distance does not account for terrain, weather, blocked roads or risk. A route that looks shortest may be unsafe or expensive, and GBFS does not guarantee the minimum-cost route. Partial updates can also make its chosen path outdated.

ii. How A* balances cost and heuristic

A* evaluates each frontier node using **f(n) = g(n) + h(n)**, where **g(n)** is the accumulated travel cost and **h(n)** estimates the remaining cost to the destination. The actual cost can represent time, energy, weather penalties and risk. Unlike GBFS, A* considers both what has already been spent and what remains. If movement costs are non-negative and the heuristic is admissible (never overestimates the true remaining cost), A* can find an optimal route in a static graph. Real-time safety still requires updated maps and route replanning.

iii. Impact of heuristic accuracy

A heuristic close to the true remaining cost generally guides both algorithms toward the goal with fewer expansions. In GBFS, the heuristic alone controls the choice, so a misleading estimate can cause large detours or unsafe choices. In A*, the heuristic influences search order but accumulated cost also matters. An overestimating heuristic may make A* faster in some cases but removes the usual optimality guarantee. Straight-line distance is useful for geometry, but it may be weak when weather, terrain and closures dominate travel time.

iv. Effect of dynamic blocked paths

When a path becomes blocked, the stored route may no longer be feasible. The drone must receive the update, mark the edge unavailable or increase its cost, and replan from its current position. Repeated changes can cause extra search work and delay. Incremental replanning methods, such as D* Lite, can reuse previous search information rather than solving the entire problem from scratch. The system should also validate the next segment before committing to it.

v. Recommendation

Use **A* with a time-and-risk-aware cost model, frequent map updates and a safety fallback**. A* is a stronger default than GBFS because it accounts for travel already incurred as well as estimated remaining cost. For rapidly changing terrain, an incremental planner such as D* Lite may be added to speed replanning. A hard safety rule should reject routes through known hazards even if they appear fast. The heuristic should be admissible when optimality is required, and the drone should replan whenever a significant closure or weather change is detected.

2. Smart-City Traffic-Signal Optimization

Formulation as an optimization problem

Let a candidate solution encode signal phase order and green-light duration for each intersection over a planning horizon. Minimize a weighted objective such as $\mathbf{J = \alpha W + \beta F + \gamma C}$, where W is total vehicle waiting time, F is fuel consumption, C is a congestion measure, and α, β and γ express the city's priorities. Subject to constraints for safe phase transitions, minimum and maximum green times, pedestrian crossing time, coordination between nearby intersections and emergency-vehicle priority. Traffic observations update the objective and constraints as conditions change.

Traditional uninformed search is inefficient because the number of possible timing combinations grows rapidly with intersections, phases and time intervals. Exhaustively enumerating candidates is impractical, and a static search may become obsolete as traffic changes.

i. Hill Climbing and its limitations

Hill Climbing starts with a feasible timing plan, evaluates neighbouring plans (for example, slightly increasing one green phase), and moves to a neighbour that improves J. It repeats until no tested neighbour improves the objective. It is simple and can improve a plan quickly, but it is local: the result depends on the starting plan and neighbourhood. It can stop before reaching the best overall solution and may oscillate if the traffic state changes while optimization is running.

ii. Local maxima, plateaus and ridges

Local optimum: A timing plan improves nearby intersections but is worse than a distant coordinated plan. No single small change appears beneficial, so the search stops.

Plateau: Many timing plans have almost equal scores, such as when a small green-time change has little measurable effect. The algorithm may wander or stop without progress.

Ridge: Improvement requires coordinated changes across several intersections. Changing only one signal at a time may not follow the direction of improvement.

iii. Simulated Annealing or Genetic Algorithms

Simulated Annealing sometimes accepts a worse move, especially early in the search, with a probability that decreases as the temperature parameter cools. This helps it escape local optima; later it becomes more selective. A Genetic Algorithm maintains a population of timing plans and uses selection, crossover and mutation to explore different combinations. Both can search beyond immediate neighbours, but they need careful parameter tuning and computational budgets. Candidate plans must be checked against safety constraints before deployment.

iv. Recommended approach

A **Genetic Algorithm combined with rolling-horizon re-optimization** is suitable when many intersections interact, because a population explores diverse coordinated plans rather than one local path. Use simulation or a fast traffic model to evaluate candidates, preserve the best feasible plan (elitism), and impose a strict runtime limit. Re-optimize when fresh traffic data arrives and retain the last safe plan if optimization misses its deadline. Simulated Annealing is a lighter alternative when computing resources are limited.

3. Autonomous Mars Exploration Rover

i. Why an online search agent is needed

Offline search assumes the environment model is sufficiently known before planning. The rover starts with an incomplete map and discovers rocks, craters, slopes and traversable ground as it moves. An online agent interleaves sensing, action and planning: it selects a safe move, observes the result, updates its knowledge and plans again. This is necessary when the full state space and terrain are not known in advance.

ii. Partial observability and dynamic conditions

Sensors have limited range and may be noisy, so a hazard can remain hidden until the rover approaches it. Terrain estimates may be uncertain, communication delays can limit remote assistance, and wheel slip or unexpected ground conditions can change the result of an action. The rover should represent uncertainty, move conservatively near unknown terrain, maintain a safe return or recovery option, and avoid treating unobserved areas as certainly safe.

iii. Building and updating an internal model

The rover combines sensor readings, localization and previous observations into a local map or occupancy grid. Each cell or terrain segment can be labelled free, blocked or unknown, with confidence or traversal cost where possible. After each action, it compares predicted and observed outcomes, updates the map and robot position estimate, and revises the plan. It should store useful observations so that already explored areas need not be rediscovered.

iv. Online search compared with uninformed and informed search

Online search: Chooses actions while learning the environment; appropriate when the map is incomplete, but may need exploration and repeated replanning.

Uninformed search: Uses the available state graph without a goal-directed heuristic. BFS can find shortest paths in unweighted graphs, but it may expand many states and requires a known graph.

Informed search: Uses a heuristic, such as estimated distance to a sample site, to focus exploration. A* can be effective on a known map, but its plan must be revised when new terrain information arrives.

v. Recommendation

Use an **online, receding-horizon planner with heuristic guidance and conservative hazard costs**. The rover should plan over the currently known local map, execute only a safe next segment, sense again and replan. Unknown cells should carry a cautious cost rather than being assumed safe. This balances adaptability and efficiency while preserving safety. If the map becomes sufficiently complete, A* can be used for longer-range route planning, with online updates handling newly discovered obstacles.

4. University Examination Timetabling as a CSP

Formulation: variables, domains and constraints

A Constraint Satisfaction Problem (CSP) is represented as **(X, D, C)**: variables X, domains D and constraints C.

Variables: For each course/exam, define its assigned time slot and hall. If an exam has special requirements, include the required room type or accommodation assignment.

Domains: The feasible time slots and available halls for each exam, after removing times when required faculty or facilities are unavailable.

Hard constraints: Exams taken by the same student cannot overlap; a hall cannot host two exams at once; room capacity must accommodate enrolled students; faculty must be available; prerequisite ordering must be respected; special accommodations must be met.

Soft constraints: Avoid too many exams for one student on the same day, minimize gaps between exams, balance hall use and avoid inconvenient schedules. These can be optimized after hard constraints are satisfied.

ii. Backtracking search

Backtracking assigns a time and hall to one exam at a time. After each assignment, it checks whether any constraint is violated. If the assignment is consistent, it continues to the next exam. If no value remains for an exam, it undoes the most recent assignment and tries another value. Choosing the most-constrained exam first (minimum remaining values) and trying the least-constraining value first can reduce backtracking.

iii. Constraint propagation and forward checking

After assigning an exam, forward checking removes incompatible time/hall choices from the domains of unassigned exams. If any domain becomes empty, the current partial schedule cannot succeed, so the algorithm backtracks immediately. Propagation therefore detects conflicts earlier and avoids exploring many impossible schedules. Stronger consistency methods can prune additional values, but require extra computation.

iv. Real-world challenges and scalable strategy

Challenges include large enrolment data, changing faculty availability, limited accessible halls, multiple campuses, late course changes and competing fairness goals. A practical strategy is CSP backtracking with minimum-remaining-values variable selection, degree-based tie-breaking, least-constraining values and forward checking. Separate hard constraints from soft preferences, use optimization to improve fairness among feasible schedules, and validate the final timetable independently. For very large instances, decomposition by campus or time block can help, provided cross-block constraints are retained.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

i. How Minimax models the game

Represent the game as a tree: nodes are game states, edges are legal moves, and terminal or depth-limited states receive utility scores. The AI is the MAX player, trying to maximize utility; the opponent is MIN, trying to minimize it. At MAX nodes, choose the largest child value; at MIN nodes, choose the smallest. This assumes a two-player, zero-sum game with an opponent who responds optimally.

ii. Computational challenges

If the average branching factor is b and search depth is d, ordinary Minimax may examine O(b^d) nodes. Even a modest increase in depth can greatly increase computation. A real-time strategy game also has many simultaneous units, actions and changing state variables. A strict deadline means the AI may be unable to search to terminal states, so it needs depth limits, a useful evaluation function and efficient move ordering.

iii. Alpha-Beta pruning with reasoning

Alpha-Beta pruning returns the same decision as Minimax when applied correctly, but skips branches that cannot affect the final choice. **Alpha (α)** is the best value MAX can guarantee along the current path; **beta (β)** is the best value MIN can guarantee. At a MAX node, update α with the largest value seen; at a MIN node, update β with the smallest value. When $\alpha \geq \beta$, remaining children of that node can be pruned because the opponent already has a choice that makes this branch no better than an available alternative. With good move ordering, the effective search can approach O(b^(d/2)) in ideal conditions; in the worst case it remains O(b^d).

iv. Evaluation function

For a non-terminal state s, one possible weighted evaluation is:

$$E(s) = 5M + 3R + 2T + 2P - 4D$$

where M is the normalized material/unit advantage, R is resource advantage, T is territory or map-control advantage, P is progress toward the objective, and D is damage or vulnerability to the opponent. Weights are illustrative and should be tuned using game simulations. All terms should be normalized to comparable scales. The evaluation must reward the actual victory condition and penalize unsafe overextension; terminal win/loss scores should dominate ordinary heuristic values.

v. Recommendation for real-time performance

Use **iterative deepening Minimax with Alpha-Beta pruning, strong move ordering and a time limit**. Iterative deepening searches depth 1, then 2, and so on, retaining the best move from the last completed depth if time expires. Search promising moves first, use a transposition table where practical, and evaluate frontier states with the weighted function. This preserves Minimax's decision logic while allowing the AI to return a valid move within the deadline. Deeper search improves play, but the system should always have a safe completed result available.